

HW5: Domain Specific Accelerator

Lin Yen-Yu
112550098

Abstract—This work designs and integrates a domain-specific accelerator (DSA) into the Aquila RISC-V SoC to accelerate CNN-based handwritten digit recognition. The accelerator is accessed through MMIO and uses Vivado floating-point IP cores to offload FMA-intensive computation in convolution, pooling, and fully-connected layers. Profiling is performed with MMIO-controlled counters to identify hotspots and quantify speedup. With buffering, ReLU offloading, and weight preloading optimizations, the advanced design achieves up to $53.96\times$ overall speedup over the software-only baseline while preserving inference accuracy, and the trade-offs between timing closure and FPGA resource utilization are discussed.

Index Terms—Domain-Specific Accelerator, RISC-V SoC, Aquila, Convolutional Neural Network, FPGA, MMIO, Floating-Point IP, AXI4-Stream

I. INTRODUCTION

In this assignment, the Aquila RISC-V SoC is used to execute a handwritten digit recognition model. Convolutional Neural Network (CNN) inference involves a large number of fused multiply-add (FMA) accumulations and frequent data transfers between the CPU core and external devices, making it computationally demanding for software-only execution.

Compiling the original C workload into an ELF binary using the RISC-V toolchain results in long inference times, motivating the integration of a custom Domain-Specific Accelerator (DSA) to improve digit recognition performance while preserving model accuracy.

This report is structured into three main stages:

- An overview of the hardware–software interface mechanisms employed, including Memory-Mapped I/O (MMIO) and floating-point IP cores from the Vivado IP Catalog.
- A description of the Domain-Specific Accelerator (DSA) module hierarchy and its control structure.
- A timing and performance analysis of accelerated CNN inference regions, followed by a discussion of bottlenecks, timing constraints, and hardware resource trade-offs in later sections.

II. HARDWARE–SOFTWARE INTERFACE

A. Memory-Mapped I/O (MMIO)

MMIO is a mechanism that allows the CPU to communicate with and transfer data to external devices through fixed memory addresses, achieving an effect similar to internal CPU load/store instructions. This makes hardware control very intuitive from software, since interacting with devices becomes just like reading and writing special memory locations.

Different devices can be selected using the `dev_sel` signal, and the handshake between the CPU and the device is completed through the `dev_ready` signal.

In the original Aquila SoC design, there are two external devices: UART and SPI IP, which are mapped to the specific MMIO address ranges showed in table I. Additionally, Aquila has reserved the MMIO region for our DSA implementation. So in my opinion, this assignment reuses these reserved address ranges as the software-accessible communication interface for triggering and polling the accelerator.

TABLE I: MMIO Address Map in Aquila SoC

Device	MMIO Address Range
UART	0xC000_0000 ~ 0xC0FF_FFFF
SPI	0xC200_0000 ~ 0xC2FF_FFFF
DSA	0xC400_0000 ~ 0xC4FF_FFFF

B. Floating Point IP (FP IP)

Floating Point IP (FP IP) is a ready-made hardware floating-point computation module provided by AMD/Xilinx in the Vivado IP Catalog. It's basically like placing a dedicated floating-point pipeline outside of the CPU core, so the SoC can directly send input vectors and receive results through AXI4-Stream interfaces, without us having to implement the floating-point datapath manually in RTL.

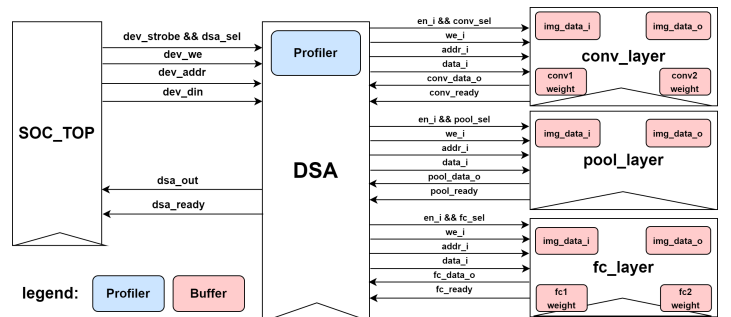
It's also convenient that the IP properties — such as latency, DSP usage, and blocking or non-blocking modes — can be configured directly through the Vivado GUI. Personally, I think this makes the timing behavior very predictable, since the pipeline delay is fixed and well-defined. In my opinion, this is a major advantage over software-emulated floating-point accumulation (software-emulated FMA), which tends to be much slower and harder to control cycle-accurately.

For this assignment, FP IP serves as the main compute core that handles the large amount of FMA operations required in both convolutional 3D dot-product accumulation and 1D dot-product accumulation in the fully-connected layer, which I feel are the most time-consuming parts of the inference pipeline.

I used the **non-blocking FMA mode with latency = 2**. I later experimented with reducing the latency to 1, and I personally think this might further improve throughput. I'll discuss these results and trade-offs more clearly in the Advanced Implementation section later.

III. DSA ARCHITECTURE

Fig. 1: My DSA Architecture



I looked into Aquila's design and, based on that, I decided to build a top module called DSA. Then, I split the hardware into three sub-modules, each implementing one of the main layer types. I personally think this structure makes it **easier to improve or swap out individual layer designs** later, and I also feel it's

simpler to maintain and debug without everything being mixed in one huge module. In my opinion, this hierarchy gives the accelerator better organization and makes long-term development less painful to manage.

IV. BASIC IMPLEMENTATION

TABLE II: Comparison of CNN_OCR Execution Time

Configuration	With FPU	Without FPU
Execute Time (ms)	7915	56333

For the following implementation, I consider the software-only execution time (Without FPU) of 56333 msec as the baseline for computing speedup. The current design with FPU runs in 7915 msec. I think the practical performance target should aim to be at least 2× faster than the FPU version, meaning the execution time should be reduced to below 3957 msec. Personally, I feel that achieving $T_{\text{accelerated}} < 3957$ msec is feasible and should serve as the main goal for evaluating accelerator efficiency in this assignment.

A. Profiler Method and Hotspot Analysis

I used MMIO registers as Profiler start/end flags to directly control the hardware counters and record execution cycles across five key regions:

- `conv_layer_fw_prop()` → 0xC4000034
- `copy_and_pad()` inside conv layer → 0xC4000038
- `convolution computing (inner FMA loop)` → 0xC400003C
- `fc_layer_fw_prop()` → 0xC4000030
- `avg_pooling_layer_fw_prop()` → 0xC4000040

Fig. 2: Example of profiling `copy_and_pad()`

```
// copy & padding
*((volatile uint32_t *)0xC4000038) = 0x1;
conv_copy_and_pad_input(entry, input);
*((volatile uint32_t *)0xC4000038) = 0x0;
```

To estimate the overhead of the activation function in the convolutional layer, I subtracted the cycle counts of regions 2 and 3 from region 1. I think this gives a reasonable approximation of the non-FMA overhead, and the results are logged in Table IV.

From the profiling window, I observed that the convolutional layer dominates about 93% of total execution time. Personally, I feel this clearly indicates the largest design headroom for improvement is in `conv_3d()`. So I believe the accelerator will benefit most from optimizing convolutional FMA throughput. In my opinion, this also means we can expect the speedup of `conv_3d()` to be the most significant in the full inference pipeline.

B. `conv_3d()`

At first, I replaced the original software accumulation logic ($A * B + C$) with a hardware-triggered FMA computation using MMIO to call the Floating-Point IP. What I personally feel is a bit special here is that, after a lot of trial and error, I found that I needed to **insert a dummy instruction** to prevent the accelerator from reading a wrong FMA result, which caused extremely low accuracy.

I think it might be related to **the hardware–software interaction latency**, or maybe the IP result arrives slightly later than I expected, but honestly, I’m still **not fully sure** why this dummy

instruction was necessary. The good news is that I later removed this workaround by applying a different optimization strategy, which I discuss in the Advanced Implementation section.

Fig. 3: `conv_3d()` basic implementation

Before:

```
for (uint32_t wyx = 0; wyx < inner_loop_iter; wyx++) {
    sum += *ppw++ * ppi[widx];
}
```

After:

```
for (uint32_t wyx = 0; wyx < inner_loop_iter; wyx++) {
    *((volatile float *)0xC400000C) = *ppw++;
    *((volatile float *)0xC4000010) = ppi[widx];
    *((volatile float *)0xC4000014) = acc;

    for (volatile int delay = 0; delay < 1; delay++); // dummy inst

    acc = *((volatile float *)0xC4000014);
}
```

C. `Fully_Connected_Layer`

Then I applied the same modification to the fully-connected layer (`fc_layer`), still using MMIO to trigger the Floating-Point IP for the FMA-based dot-product computation. Similar to what happened in `conv_3d()`, I personally found that inserting a dummy instruction was necessary to avoid a major accuracy drop.

I think this dummy instruction acted like a safeguard, because even though the loop break condition was 0, skipping this extra instruction would still lead to incorrect FMA results being sampled, and the model **accuracy became extremely low**. To be honest, this fix is something I reached only **after a lot of trial and error**, and I still feel I don’t fully understand the real root cause yet. Fortunately, I removed this workaround later — in the Advanced Implementation section.

Fig. 4: `fc_layer` basic implementation

Before:

```
for (uint32_t c = 0; c < entry->base.in_size; c++)
    a[i] += w[i*entry->base.in_size_ + c] * in[c];
```

After:

```
float acc = 0.0f;
for (uint32_t c = 0; c < entry->base.in_size; c++) {
    *((volatile float *)0xC4000000) = w[i * entry->base.in_size_ + c];
    *((volatile float *)0xC4000004) = in[c];
    *((volatile float *)0xC4000008) = acc;

    for (volatile int delay = 0; delay < 0; delay++); // useless dummy inst

    acc = *((volatile float *)0xC4000008);
}
a[i] = acc;
```

TABLE III: Profiler Cycle Region Time and Speedup Comparison

Region	Baseline		Basic		Speedup
	Time (ms)	Ratio (%)	Time (ms)	Ratio (%)	
Copy_and_Pad	78.99	0.14	78.88	0.66	x1.00
Conv_Compute	51,788.08	91.93	8,919.17	75.15	x5.81
Conv_Activate	646.58	1.15	649.50	5.47	x1.00
Conv_Total	52,513.65	93.22	9,647.56	81.29	x5.44
FC_Total	1,007.34	1.79	1,207.51	10.17	x0.83
Pooling_Total	2,812.26	4.99	1,013.12	8.54	x2.78
Sum	56,333.26	100.00	11,868.18	100.00	x4.75

From Table IV, I observed that the Basic implementation achieves an overall speedup of **x4.75**. I think the most noticeable improvement comes from *Conv_Compute*, which shows a significant speedup of **x5.81**. This result matches my expectation, since convolution computation is the dominant hotspot in the original software baseline.

However, I personally find it a bit confusing that the *FC_Total* region becomes slower (only **x0.83**), while *Pooling_Total* unexpectedly becomes faster (**x2.78**). To be honest, I still do not fully understand the root cause of this behavior. In my experience, this phenomenon also occasionally appears in the Advanced Implementation results, so I believe it might be related to measurement windows, data movement/handshake timing, or how the profiling flags are placed.

Overall, I think the Basic implementation is a solid step forward, but it still cannot meet the target performance goal of **3957 msec**. Therefore, the next step is to further tune the design and experiment with different configurations to push the execution time down toward the target.

V. ADVANCED IMPLEMENTATION

A. Average Pooling Layer

First, I think the pooling layer can also be accelerated by dedicated hardware modules, but unlike convolution or fully-connected layers, it requires inserting additional IP blocks instead of reusing a single FP core. In this CNN model, the pooling layer essentially needs two major computation steps:

- Step 1: Compute the sum of elements in a 2x2 kernel window. I think this can be done using the ADD IP.
- Step 2: Compute the average of the summed result. I personally feel this can be implemented using the MUL IP.

In my opinion, splitting the pooling computation this way keeps the design modular, and I believe it still has enough performance headroom for improvement even though it is not the main bottleneck.

B. FP IP Latency

From the original slides (Slide 22), the default FP IP latency was 17 cycles, which I think means that every FMA operation needs to wait 17 cycles before the result becomes available. I feel this has a huge impact on both convolution and fully-connected layers, because the nested accumulation loops stall frequently while waiting for results.

So I think I needed to optimize this delay, and I adjusted the FP IP latency down to 2 cycles, which, in my opinion, stays within a safe timing margin without introducing timing violations. I personally applied the same latency modification to the ADD IP and MUL IP I inserted for the pooling layer as well. I believe this helps maintain predictable hardware–software interaction timing while significantly improving pipeline throughput for all FMA-related workloads in this accelerator design.

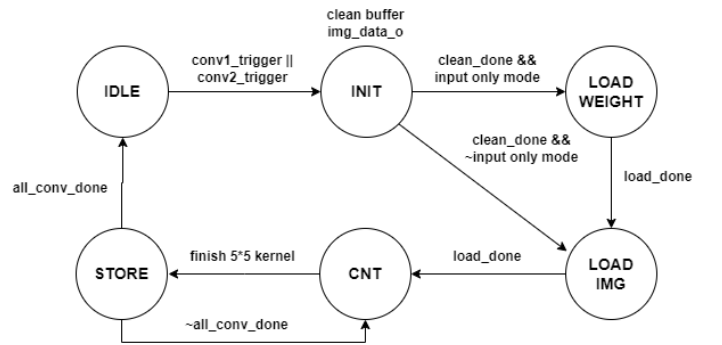
C. Buffering

I personally think this is the most important architectural change I made. In the earlier convolution design, the DSA was triggered every time an FMA operation was executed. I feel this means each computation step suffered from repeated MMIO communication and index calculation overhead, which I think limited the real acceleration benefit.

After reviewing Slide 27, I believed this bottleneck could be improved by buffering data on-chip in the FPGA. So, in my opinion, the idea was to load the entire input image and all neural weights onto the FPGA board first, store them in input buffers, and then let the hardware FSM handle both the computation stages and sliding-window index generation. I personally think this allows the accelerator to perform sliding-window convolution and accumulate results into the output image buffer internally, and finally emit the completed convolution image back to software in one single MMIO readout instead of many repeated triggers.

To support this flow, I designed the following FSM structure:

Fig. 5: Convolution Layer FSM

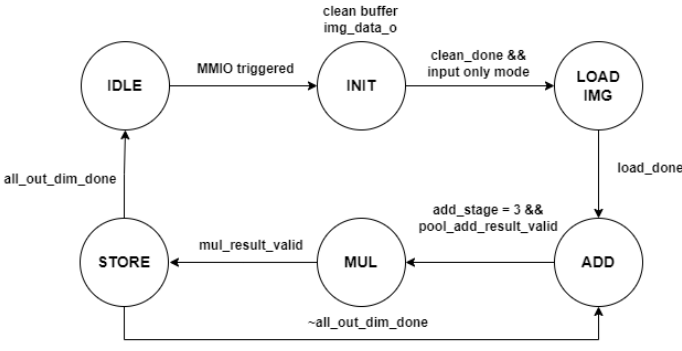


Each state's role, as I currently understand it, is summarized below:

- **S_IDLE (0)**: I think this is the default waiting state, where the accelerator stays until software writes '1' to the trigger register.
- **S_INIT (1)**: Initializes the output buffer to zero. I personally feel this guarantees a clean start for accumulation.
- **S_LOAD_WEIGHT (2)**: Loads convolution weights into internal buffers like 'conv1_weight_data' or 'conv2_weight_data'. I think this mimics model program loading.
- **S_LOAD_IMG_I (3)**: Loads the input image into the on-FPGA image buffer. I feel this stage dominates the data-feeding cost.
- **S_CNT (4)**: Performs the 3D dot-product FMA accumulation for sliding-window convolution. In my opinion, this is the main hotspot for hardware acceleration.
- **S_STORE (5)**: Writes the accumulated result into the output image buffer. I think this works like committing a pipeline result.

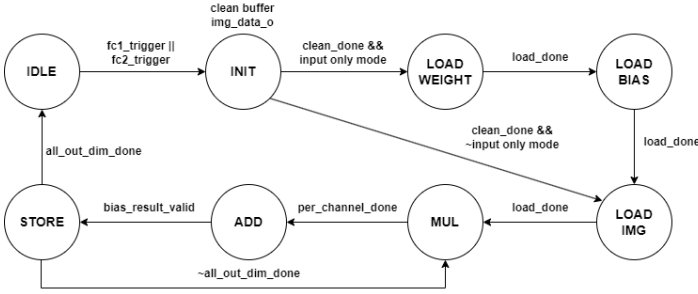
I also built the pooling layer and fully-connected layer buffering architectures using the same convolution FSM hierarchy I designed earlier. I personally think that since the overall structure is identical, repeating the detailed responsibilities of every state would be redundant, so I decided not to go into them again.

Fig. 6: pool_layer FSM



For the pooling layer, as I mentioned before, the computation happens in two stages — summation and averaging. The original CNT state needed to be split, so I decomposed it into ADD and MUL states. Because pooling uses a 2×2 sliding window, I feel that the accumulation loop is relatively small, and the loop itself contains about 3 internal FSM states, which I think is enough to finish $\text{ADD} \rightarrow \text{MUL} \rightarrow \text{STORE}$ style averaging safely without violating timing assumptions.

Fig. 7: fc_layer FSM



In the fully-connected layer, I needed one extra state, `LOAD_BIAS`, because when I looked into the CNN source code, I noticed that Layer 5’s FC forward propagation adds a bias term after the dot product. I personally feel that without explicitly loading the bias into the FPGA, the hardware cannot reproduce the exact computation behavior of the software reference. So, in my opinion, I also buffered the bias values into the FPGA at the same time as the weights, letting the FSM start from bias-aware initialization and match the original computation more accurately.

D. ReLU

After reviewing `cnn_ocr.c`, I noticed that every convolutional layer activation call uses ReLU. I personally think this makes ReLU a perfect candidate for a lightweight hardware implementation, because the behavior is simple and deterministic.

From a hardware perspective, I feel that we only need to check the 31st bit of the 32-bit result (the sign bit) to know whether the output should be 0 or the original value. So in my opinion, the ReLU logic can be implemented purely as:

$$\text{ReLU}(x) = (x[31] == 1) ? 0 : x$$

Because of this, I decided to modify the design so that the accelerator writes the ReLU output directly into the MMIO output buffer `out[]` (e.g., `0xC400_0040` region), instead of producing a temporary vector `a[]` first and then copying it back to `out[]` in software. Personally, I feel this removes one full memory copy and reduces unnecessary MMIO traffic, which I think makes the hardware–software interaction cleaner and faster.

So I believe this direct-write ReLU design eliminates the need for the software post-processing stage, making the conv layer activation path more efficient while keeping timing predictable for the pipeline.

E. Weight Loading Optimization

When evaluating the model, I need to run CNN inference on 100 input images. I think the key observation here is that the only difference between these 100 images is the input pixel values — the neural network weights are exactly the same. So in my opinion, reloading the same weights every time is unnecessary and just increases MMIO traffic.

Because of this, I decided to optimize the design so that the accelerator performs `LOAD_WEIGHT` only once, during the first evaluation. After forward propagation finishes, I let a hardware flag `weight_preloaded` rise, which I personally feel acts like a “weights are ready” indicator. This means the remaining 99 images can skip the weight loading stage entirely. I applied this optimization to both the convolutional layer and the fully-connected layer FSMs.

For example, in Layer 5’s fully-connected stage, the original software would request weight loading for every input image. Since a single FC call takes a $4 \times 4 \times 12$ feature map and multiplies it with 30 weights, I think the total number of weight loads for 100 images becomes:

$$(4 \times 4 \times 12) \times 30 \times 100 = 576,000 \text{ times}$$

After my optimization, I only need to load the weights once:

$$(4 \times 4 \times 12) \times 30 = 5,760 \text{ times}$$

I believe that by reducing redundant MMIO transactions, the accelerator spends fewer cycles waiting for data movement, which I personally think increases the effective computation throughput and makes the hardware–software interaction much lighter during repeated inference.

VI. ANALYSIS OF ACCELERATOR

A. Timing Analysis

The speedup is calculated by dividing the execution time of each region by the software-only baseline time. From the table, I personally observed an overall speedup of **x53.96**. What I find really interesting is that the `Conv_Compute` region alone even reached around **x68.12** speedup. The second most significant acceleration comes from the fully-connected layer, which I feel achieved about **x14.80** speedup. After accelerating ReLU, I also noticed that the `Conv_Activate` region shows around **x8.60** speedup, which I think is already a meaningful improvement for activation overhead.

However, even after these optimizations, I still feel that the main bottleneck layer remains the convolutional layer. So I believe that if we want further acceleration, we should still prioritize improving `Conv_Compute` throughput. From my own perspective, convolution computation still dominates the runtime share, so in my opinion, accelerating the FMA parallelism or data-feeding efficiency of the convolution core should again be the top focus in the next design iteration.

TABLE IV: Profiler Cycle Region Time and Speedup Comparison

Region	Baseline		Basic		Advanced	
	Time (ms)	Speedup	Time (ms)	Speedup	Time (ms)	Speedup
Copy_and_Pad	79.00	x1.00	78.88	x1.00	75.46	x1.05
Conv_Compute	51,788.08	x1.00	8,919.17	x5.81	760.24	x68.12
Conv_Activate	646.58	x1.00	649.50	x0.83	75.52	x8.60
Conv_Total	52,513.65	x1.00	9,647.56	x5.44	835.77	x11.54
FC_Total	1,007.34	x1.00	1,207.51	x0.83	81.58	x14.80
Pooling_Total	2,812.26	x1.00	1,013.12	x2.78	126.56	x8.01
Sum	56,333.26	x1.00	11,868.18	x4.75	1,043.90	x53.96

B. Resource Analysis

TABLE V: FPGA Resource Utilization and Efficiency Comparison

Resource	Aquila Max	Baseline	Advanced
LUTs	63,400	7.28%	55.48%
LUT (logic)	63,400	7.14%	34.88%
LUT (mem)	19,000	0.45%	68.73%
D Flip-Flop	126,800	3.07%	18.93%
BRAM	135	12.59%	33.70%
DSP	240	1.67%	12.50%

After inserting FP IPs, I observed that the DSP usage increased **from 4 to 30**. I think this is because the floating-point IP blocks internally map multiply, add, and other arithmetic stages to DSP48 slices, and I personally feel this aligns with what the slides implied — using external FP pipelines trades hardware resources for speed.

I also noticed that LUT-based memory (LUT-mem) and BRAM utilization grew a lot. In my opinion, this is mainly because I buffered the full CNN weights and the entire input image set on the FPGA side, instead of streaming them repeatedly from software. I think this reduces MMIO transactions, but it also means the FPGA has to allocate real storage for those tensors, which I believe naturally increases BRAM and LUT-mem demand.

So overall, I personally think the 53.96 $\times$ speedup came with a non-trivial cost in Aquila’s available FPGA resources. I feel this is acceptable for this small CNN model, but I think if the same idea is applied to larger CNNs or real foundation models later, we’ll definitely need to consider the **resource-performance trade-off** more seriously. In my opinion, this kind of co-design always ends up being a balancing act between hardware headroom and achievable acceleration, and I think that will be an important challenge for future work.

VII. DISCUSSION

A. Timing Issue

The slides stated that latency = 2 is the minimum latency setting that does not violate timing requirements on the target FPGA. I thought it might still be possible to push the latency down to 1, so I tried synthesizing the circuit with latency = 1.

After doing that, I observed a total execution time of 801 ms, which I think corresponds to an overall **x70.33** speedup compared to the software-only baseline. But the synthesis timing report showed **WNS < 0**, which, in my understanding, means the critical path cannot finish within a single clock cycle at the original clock rate.

At first, I guessed that maybe only a few paths would violate timing, but after checking the full timing report, I personally found that most paths inside the `conv_layer` datapath failed timing, not just one or two. I think this indicates that the convolution core simply cannot meet the clock constraint when forced into latency = 1. So in my opinion, the only realistic fix was to reduce the clock frequency to relax the timing constraint and make the design synthesizable.

Fig. 8: Latency = 1’s WNS

Status	WNS	TNS
synth_design Complete!		
write_bitstream Complete!	-11.858	-3513.036

B. DSP

I noticed that the Floating Point IP configuration panel in Vivado provides different DSP usage optimization levels, such as medium or full usage. I remembered that in Lab1, I used 4 DSPs for the mul/div unit, and I think that gave a clear performance improvement there. So I wanted to see if the same idea could help this time.

Because of that, I tried increasing the FP IP DSP usage from **Medium Usage (2 DSPs)** to **Full Usage (4 DSPs)** for all FMA-based blocks in both convolution and fully-connected layers. After synthesizing again, I observed that the overall speedup only **changed from x53.96 to x53.98**, which I personally feel is basically no improvement at all.

So I think this DSP scaling does not help remove the real bottleneck, which, in my opinion, is not the DSP count but the overall data-feeding efficiency and the true pipeline utilization of the convolution core. Because the improvement was almost zero, I decided not to include this experiment in my final performance analysis or results section.

VIII. FUTURE WORK

A. Scalability and Portability Considerations

In this assignment, I did not add specific handling for zero padding or stride in hardware, and I also did not parameterize kernel size or input width/height. I think this limits the accelerator’s ability to be reused on other CNN models. Personally, I feel that to support model portability, all adjustable parameters should be configured centrally through a unified specification or configuration file. In my opinion, only after this level of abstraction is introduced can the accelerator have real potential to be migrated and applied to different CNN architectures in the future.

B. FMA Parallel Computation

Because the convolutional layer and the fully-connected layer both require a huge amount of FMA computations, I think having only one FMA unit in each layer really limits the accelerator's throughput. I personally feel that if multiple FMA units with latency = 2 can be used at the same time and run in parallel, we might be able to finish several operations within about 3 cycles and **achieve doubled efficiency**. I know this would cost extra DSPs, BRAM, and LUTs, but from Table 2, it looks like there are still some remaining hardware resources that we can use. So I think, or I would even say I feel, that this idea should still be feasible in this design.

C. Integration between Layers

From the profiler, I noticed that the overhead between layers is actually very small — around 6ms, which I think is almost negligible for this model. But I feel that in deeper CNNs like ResNet-50 or MobileNet, where there are many more layers, the MMIO overhead might start to add up. So I personally think that combining two adjacent layers into one hardware invocation could reduce repeated MMIO triggers and further save communication overhead. In my opinion, this kind of layer fusion might be a practical direction if we want the design to scale to real multi-layer CNN workloads.

IX. CONCLUSION

This homework was honestly one of the most fun and satisfying assignments I've done. I feel a strong sense of achievement from trying out so many different ways to accelerate the model, especially when I unexpectedly found new optimization opportunities in places I didn't initially consider — those moments really felt like breakthroughs.

I think the biggest takeaway for me is learning how to properly design FSMs and structure hardware modules in a scalable and maintainable way. I believe this experience is going to be very useful for my future hardware–software co-design work.

After this, I feel like this project deserves to be cleaned up and reorganized into a standalone side project. I think I'll continue exploring whether I can squeeze.

REFERENCES

- [1] Vivado Ram style
https://docs.amd.com/r/en-US/ug901-vivado-synthesis/RAM_DECOMP-XDC-Example